

Object Oriented Programming
(CS1004)

Course Instructor(s):

Mr. Shehreyar Rashid, Mr. Aqib Rehman, Ms. Munazza
Niazi, Ms. Momina Behzad

Section(s): CS(A,B,C,D,E,F,G,H,J)

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 60

Total Questions: 3

Date: Apr 15, 2026



Roll No



Course Section



Student Signature

Do not write below this line.

Attempt all the questions.

Instruction:

1. Attempt all questions on given space in question paper. No answer on rough sheet will be marked.
2. No overwriting or cutting allowed
3. Read questions carefully.

[CLO 3: Model an algorithmic solution for a given problem using OOP]

Q1: You are designing a Disaster Simulation System where each Country stores Resources and Agent in a dynamic array. You are required to design and implement the given classes in C++, where following code should run without any syntax/logical errors. GIVEN MAIN FUNCTION (DO NOT MODIFY).

[30 marks]

```
int main()
{
    Country c1(2,1); // country with 2 resources, 1 agent
    Country c2(1,1);
    Resource r1(10), r2(20), r3(30);
    Agent a1(101), a2(202);
    c1[0] = r1;
    c1[1] = r2;
    c2[0] = r3;
    c1 += a1; // add agent
    Country c4(++ a2);
    Country c3 = c2 - r1; // remove resources
    cout << c3; // display
    return 0;
}
```

Q1 : 08

Q2 : 05

Q3 : 12

Expected Output:

Country c1 is created with 2 resources and 1 agent capacity.
 Country c2 is created with 1 resource and 1 agent capacity.
 Resource r1 is created with value 10.
 Resource r2 is created with value 20.
 Resource r3 is created with value 30.
 Agent a1 is created with id 101.
 Agent a2 is created with id 202.
 Resource r1 is assigned to c1.
 Resource r2 is assigned to c1.
 Resource r3 is assigned to c2.
 Agent a1 is added to c1.
 Country c4 has Agent a2 (id = 203).
 Country c3 has resources by removing resource r1 from c2.
 Country c3 data:
 Resource(s): 30
 Agent(s): (existing agent of c2, if maintained properly)

//Resource Class

class Resource

{

// define data members here

int value;
 static int resources

// write only the prototype of required

//functions here

Resource (int);

//Agent Class

class Agent

{

// define data members here

int id;
 static int agents;

// write only the prototype of required

//functions here

Agent (int);

};

//Country Class

class Country

{

// define data members here

~~Resource **r;~~ Resource *r;

~~Agent **a;~~ Agent *a;

~~Agent **a;~~ static int
countries;

~~static int countries;~~

2/

// write only the prototype of functions here

Country (int , int);

1/

};

National University of Computer and Emerging Sciences
Islamabad Campus

Q/F

Country*

//Write the operator overloading functions here

Country ::operator [] (int index) {

return Resou;

return

r

}

void Country ::operator += (Agent & a) {

cout <<

Print Agent ::operator ++ (

)

}

Ostream & ::operator << (ostream o, Country & c

)

}

Diagram illustrating a hierarchical structure with nested boxes and labels:

- Top level: `::operator` ()
- Second level: `}`
- Third level: `::operator` ()
- Bottom level: `}`

//Any other method's which is required to run the main function

```
Country(int r, int a){
    r = new Resource * [r];
    a = new Agent * [a]; countries++; resources += r; agents += a;
    cout << "Country C " << countries << " is created with " << resources << " resources "
    << " and " << agents << endl;
}
```

[CLO 1: Demonstrate the basic concepts of OOP]

Q2: Trace the following code snippets and write their output. If code contains any error then identify the error and write complete explanation for that error. [15 marks]

CODE	OUTPUT OR ERROR
<pre>#include <iostream> using namespace std; class Magic { private: long long int *n, size; public: Magic(int s) : size(s), n(new long long int[size]) { size=s; for(int i=0; i<size; i++) n[i]=i+1; } void display(){ for(int i=0; i<size; i++) cout<<n[i]<<" "; } ~Magic() { delete [] n; } }; int main(){ Magic magic(5); magic.display(); return 0; }</pre>	1 2 3 4 5 no error

```

#include <iostream>
using namespace std;
class Vector;
class Matrix {
    int arr[3];
public:
    Matrix() { for(int i=0;i<3;i++)
arr[i]=i+100; }
    int operator[](int i) { return arr[i]; }
    operator Vector();
};
class Vector {
    int data[3];
public:
    Vector(Matrix m) {
        for(int i=0;i<3;i++) data[i] = m[i];
    }
    int operator()(int i) { return data[i]; }
};
Matrix::operator Vector() {
    return Vector(*this);
}
int main() {
    Matrix m;
    int result = Vector(m)(m[1]);
    cout << result;
    return 0;
}

```

101

```

#include <iostream>
using namespace std;
class Engine {
public:
    Engine() { cout << "Engine Constructor\n"; }
    ~Engine() { cout << "Engine Destructor\n"; }
};
class Calculator {
    int value;
    Engine e;
public:
    Calculator(int v = 1) : value(v) {
        cout << "Calculator Constructor\n";
    }
    Calculator* add(int x) {
        value *= x;
        return this;
    }
    Calculator* multiply(int x) {

```

Engine Constructor
 Calculator Constructor
 Wrapper Constructor
 Final value : 7
 Wrapper Destructor
 Calculator Destructor
 Engine Destructor

05

```
        value += x;
        return this;
    }
    Calculator* subtract(int x) {
        value -= x;
        return this;
    }
    void display() {
        cout << "Final Value: " << value <<
endl;
    }
    ~Calculator() {
        cout << "Calculator Destructor\n";
    }
};
class Wrapper {
    Calculator* c;
public:
    Wrapper(Calculator* c1) : c(c1) {
        cout << "Wrapper Constructor\n";
    }
    void process() {
        c->add(3)->multiply(2)->subtract(1)-
>display();
    }
    ~Wrapper() {
        cout << "Wrapper Destructor\n";
    }
};
int main() {
    Calculator obj(2);
    Wrapper w(&obj);
    w.process();
    return 0;
}
```

[CLO 2: Apply OOP concepts(Encapsulation, Inheritance, Polymorphism, Abstraction) to computing problems for the related program.]

Q3: CryptoQuest is an entertainment company that offers themed escape room experiences. Each EscapeRoom is characterized by attributes such as name, theme, and maximum player capacity. An escape room is composed of multiple Puzzle objects that are physically integrated into its structure; therefore, the lifecycle of puzzles is strictly dependent on the room. If an escape room is removed, all associated puzzles are destroyed as well. Each Puzzle contains attributes including description, points, and difficulty level. Puzzles are further specialized into two categories: LogicPuzzle, which includes

additional attributes such as riddle text and answer key, and PhysicalPuzzle, which specifies a mechanism type (e.g., lever, rotating lock). Every escape room is equipped with a dedicated TimerSystem responsible for tracking session duration and remaining time. The timer system is tightly coupled with its respective room and cannot exist independently. The company employs individuals categorized under the abstract entity Person, defined by attributes such as name, age, and email. Two specialized roles inherit from Person: Player and GameMaster. A Player has additional attributes including nickname and games-played count, while a GameMaster possesses an experience level and can be assigned to supervise multiple escape rooms. GameMasters exist independently of any specific room. Players can form Teams, each identified by a unique team name. Players maintain independent existence and can freely join or leave teams without affecting their lifecycle.

Requirements: Identify all classes, key attributes, at least one method per class, and all relationships. **DRAW UML DIAGRAM** to demonstrate all the identified classes, attributes, operations and relationships.

[15 marks]

